

Compte rendu n°2

Tp Tests de Logiciels

Nom : Rayen Bouyahia

Classe : L2CS01

1. Introduction

Ce TP illustre l'approche TDD (Test-Driven Development) appliquée à une classe Java représentant un loup doté d'une orientation. Le principe consiste à écrire le test avant d'implémenter le code, puis à faire évoluer progressivement l'implémentation jusqu'à ce que tous les tests passent.

Les étapes abordées dans ce TP sont :

- Création du premier test unitaire vérifiant la position initiale du loup (NORD).
- Mise à jour de la classe Loup pour faire passer le test.
- Ajout d'un deuxième test vérifiant la rotation du loup.
- Implémentation de la méthode tourner() pour valider les deux tests.
- Remaniement (refactoring) du code tout en conservant les tests au vert.

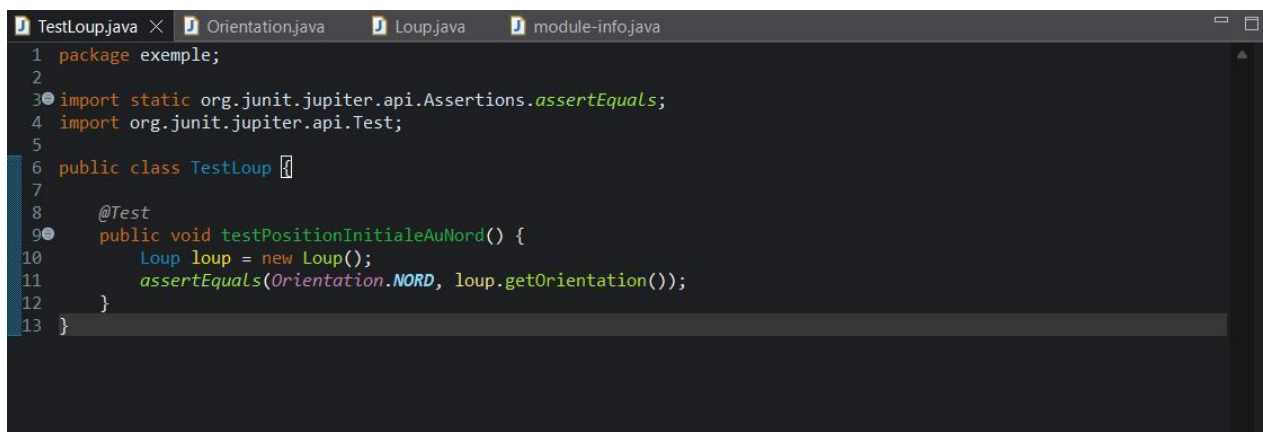
2. Création du premier test (position initiale du loup)

2.1 Présentation

La première étape consiste à écrire un test unitaire qui vérifie que la position initiale du loup est bien orientée vers le NORD. En TDD, on écrit ce test à partir d'une classe Loup vide, ce qui provoque un échec attendu.

2.2 Classe de test initiale — TestLoup.java

Le test créé instancie un Loup et vérifie son orientation via assertEquals :



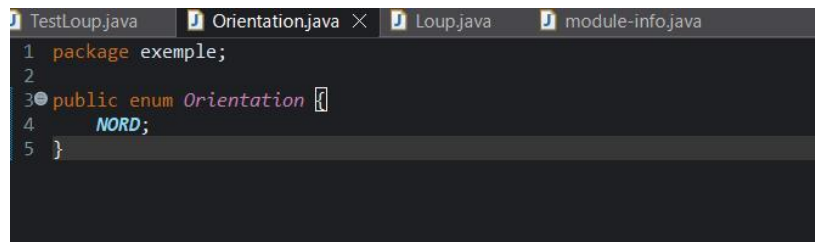
```
1 package exemple;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import org.junit.jupiter.api.Test;
5
6 public class TestLoup {
7
8     @Test
9     public void testPositionInitialeAuNord() {
10         Loup loup = new Loup();
11         assertEquals(Orientation.NORD, loup.getOrientation());
12     }
13 }
```

Fig. 1 — Classe TestLoup.java avec le premier test

2.3 Enum Orientation.java

Pour compiler le test, une énumération Orientation est créée avec une seule valeur initiale :

NORD.

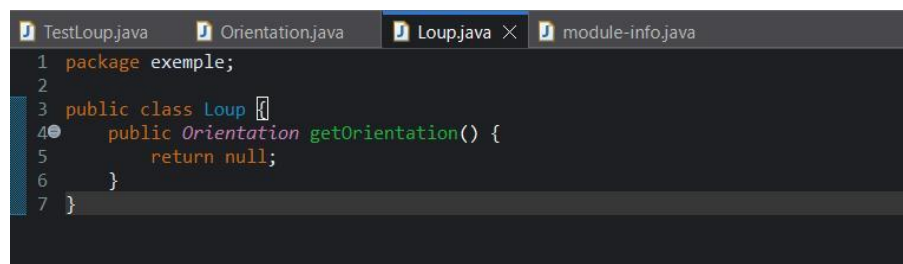


```
1 package exemple;
2
3 public enum Orientation {
4     NORD;
5 }
```

Fig. 2 — Enum Orientation.java avec la valeur NORD

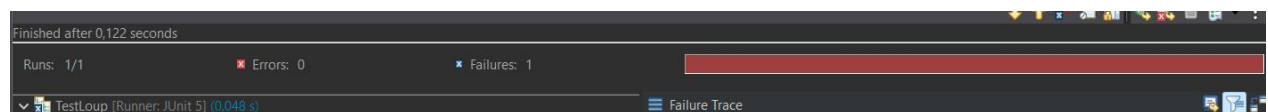
2.4 Squelette initial de la classe Loup.java

La classe Loup est créée avec un squelette minimal : la méthode getOrientation() retourne null, ce qui provoque l'échec du test (Failures: 1).



```
1 package exemple;
2
3 public class Loup {
4     public Orientation getOrientation() {
5         return null;
6     }
7 }
```

Fig. 3 — Classe Loup.java — squelette initial (getOrientation retourne null)



Finished after 0.122 seconds

Runs: 1/1 Errors: 0 Failures: 1

TestLoup (Runner: JUnit 5) (0.048 s)

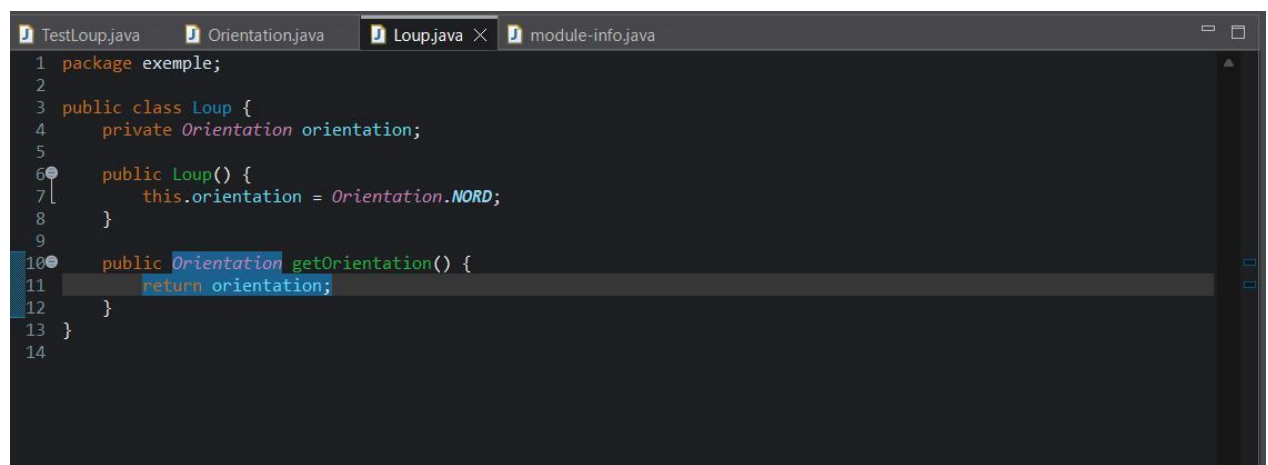
Failure Trace

Fig. 4 — Résultat JUnit : Runs 1/1, Errors 0, Failures 1 — test échoué (rouge)

3. Correction de la classe Loup pour faire passer le test

3.1 Mise à jour de Loup.java

Pour que le test passe, la classe Loup est modifiée : un attribut orientation est ajouté, initialisé à Orientation.NORD dans le constructeur, et retourné par getOrientation().



```
1 package exemple;
2
3 public class Loup {
4     private Orientation orientation;
5
6     public Loup() {
7         this.orientation = Orientation.NORD;
8     }
9
10    public Orientation getOrientation() {
11        return orientation;
12    }
13 }
14
```

Fig. 5 — Classe `Loup.java` après correction : constructeur + `getOrientation()`

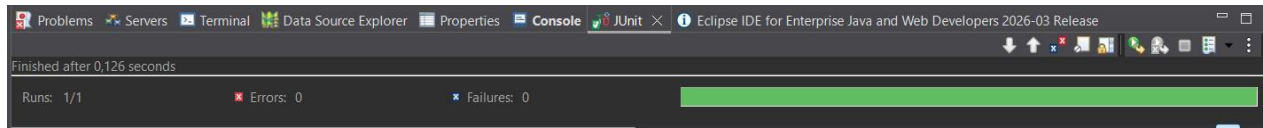


Fig. 6 — Résultat JUnit : Runs 1/1, Errors 0, Failures 0 — test réussi (vert)

4. Test de `testPositionInitialeAuNord()` et ajout de `testTournerUneFois()`

4.1 Nouvelle classe de test `TestOrientation.java`

Une deuxième classe de test est créée. Elle regroupe deux tests : le premier vérifie la position initiale (NORD), le second vérifie qu'après un appel à `tourner()`, le loup fait face à l'EST. Comme la méthode `tourner()` n'existe pas encore, une erreur de compilation est détectée.

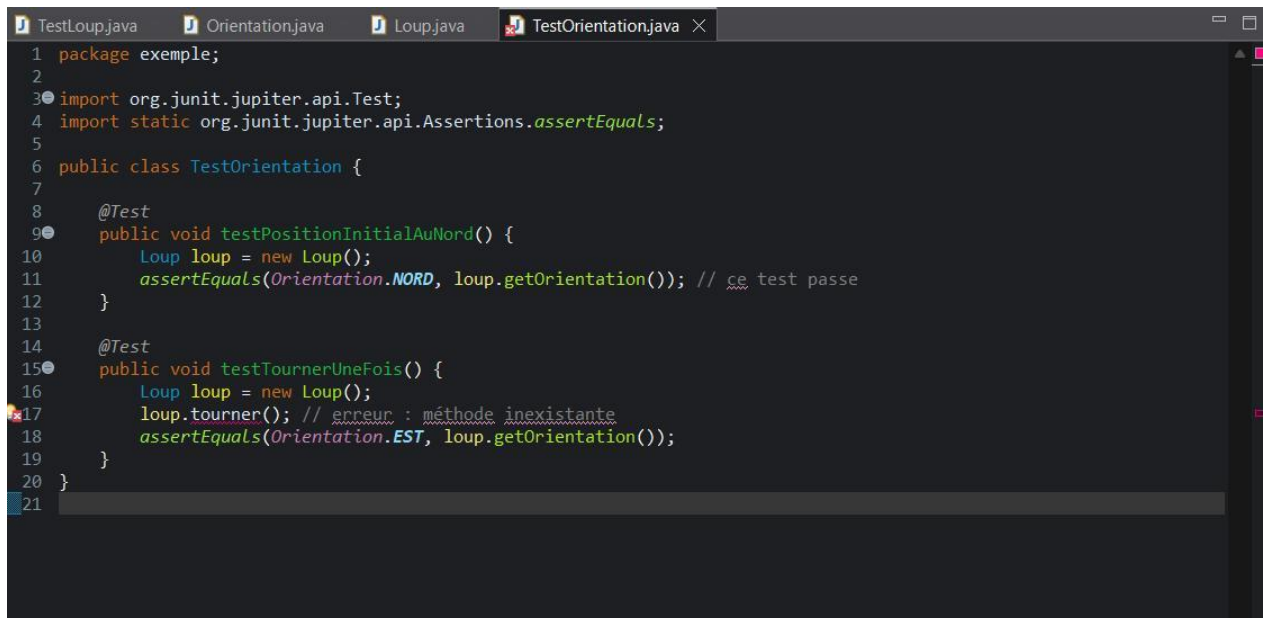


Fig. 7 — `TestOrientation.java` : `testPositionInitialeAuNord()` + `testTournerUneFois()` — erreur méthode inexistante

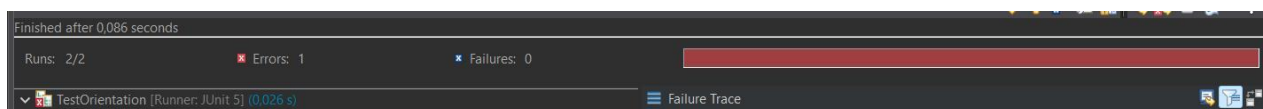
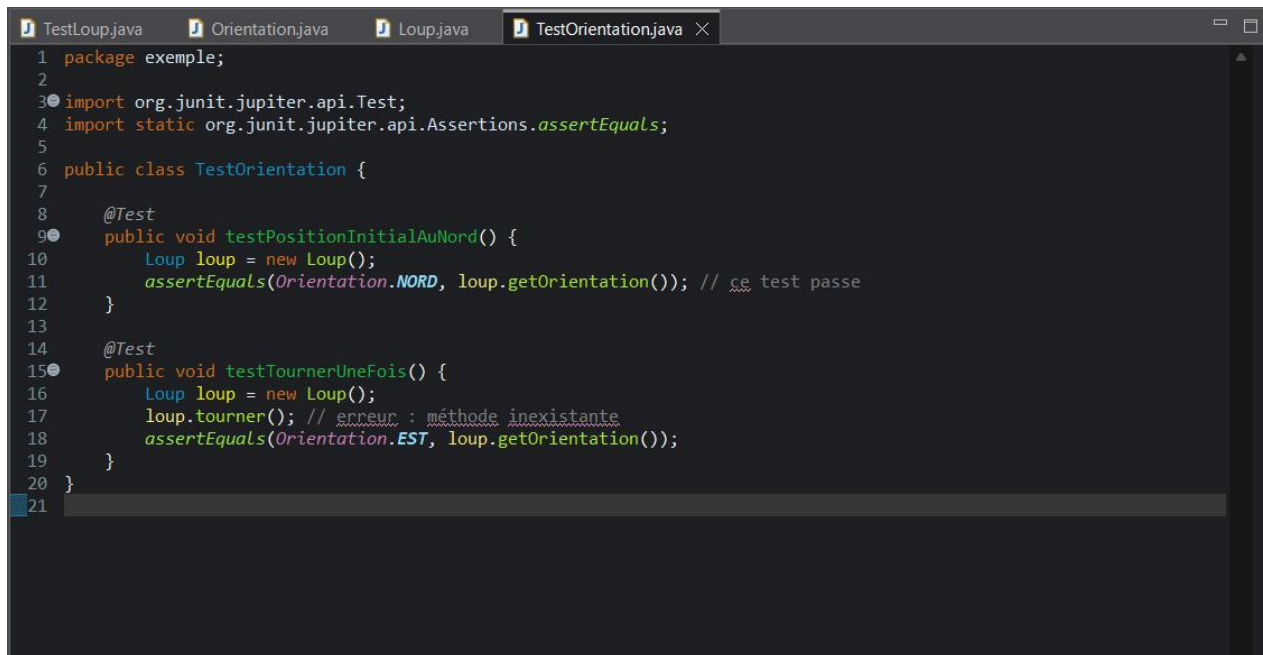


Fig. 8 — Résultat JUnit : Runs 2/2, Errors 1, Failures 0 — erreur sur `tourner()`

5. Implémentation de la méthode `tourner()`

5.1 Première implémentation

La méthode `tourner()` est implémentée dans la classe `Loup`. Elle met à jour l'orientation du loup selon la séquence Nord → Est → Sud → Ouest → Nord. Après implémentation, les deux tests passent.



```
1 package exemple;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5
6 public class TestOrientation {
7
8     @Test
9     public void testPositionInitialAuNord() {
10         Loup loup = new Loup();
11         assertEquals(Orientation.NORD, loup.getOrientation()); // ce test passe
12     }
13
14     @Test
15     public void testTournerUneFois() {
16         Loup loup = new Loup();
17         loup.tourner(); // erreur : methode inexistant
18         assertEquals(Orientation.EST, loup.getOrientation());
19     }
20 }
21
```

Fig. 9 — TestOrientation.java avec les deux tests après implémentation de tourner()



Fig. 10 — Résultat JUnit : Runs 2/2, Errors 0, Failures 0 — tous les tests réussis (vert)

6. Remaniement du code (Refactoring)

6.1 Principe du refactoring

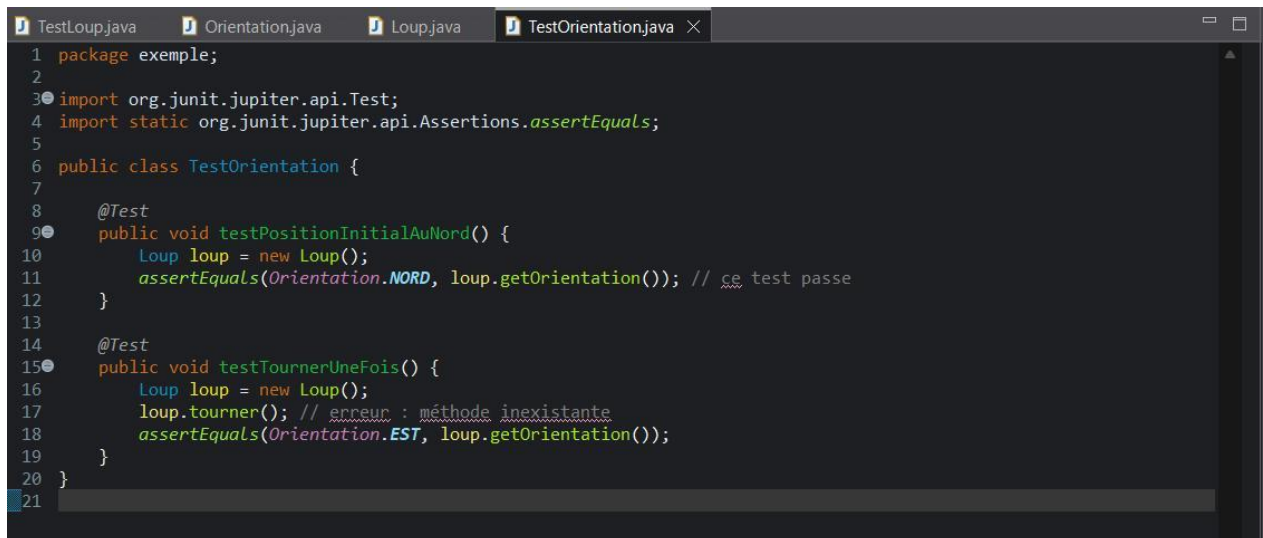
Après que tous les tests passent, le code est retravaillé pour améliorer sa lisibilité et sa maintenabilité, sans modifier le comportement observable. Les tests servent de filet de sécurité : si un test échoue après remaniement, cela signale une régression.

Les actions de remaniement typiques incluent :

- Suppression du code dupliqué.
- Amélioration du nommage des variables et méthodes.
- Simplification de la logique conditionnelle (ex. utilisation de Enum.values() et modulo).

6.2 Code remanié et résultat

Après remaniement, les tests sont relancés pour confirmer que le comportement est inchangé. Les deux tests passent toujours.



```
1 package exemple;
2
3 import org.junit.jupiter.api.Test;
4 import static org.junit.jupiter.api.Assertions.assertEquals;
5
6 public class TestOrientation {
7
8     @Test
9     public void testPositionInitialAuNord() {
10         Loup loup = new Loup();
11         assertEquals(Orientation.NORD, loup.getOrientation()); // ce test passe
12     }
13
14     @Test
15     public void testTournerUneFois() {
16         Loup loup = new Loup();
17         loup.tourner(); // erreur : methode inexistante
18         assertEquals(Orientation.EST, loup.getOrientation());
19     }
20 }
21
```

Fig. 11 — TestOrientation.java après remaniement du code



Fig. 12 — Résultat JUnit : Runs 2/2, Errors 0, Failures 0 — tous les tests toujours au vert

7. Bilan & Points clés

Ce TP a permis de mettre en pratique le cycle TDD complet (Rouge → Vert → Refactor) :

- Rédaction du test avant le code : le test échoue d’abord (rouge), ce qui valide sa pertinence.
- Implémentation minimale pour faire passer le test : objectif « vert ».
- Remaniement sans régression : le code est amélioré tout en conservant les tests au vert.
- L’utilisation d’un enum Java (Orientation) renforce la sécurité de type.
- La méthode assertEquals est utilisée pour vérifier l’état interne de l’objet.